

Vulnerability Management Policy

Parent: INFOSEC_POLICY.md **Owner:** Matt Fellows — matt@tangiblevalue.com **Effective date:** 2026-05-06 **Review cadence:** annually, or on material change to the dependency or runtime stack.

1. Scope

What we monitor for vulnerabilities, in priority order:

Asset	What we look for	Tool / Source
Application dependencies (package.json × 2)	Known CVEs in npm packages and their transitive deps	Dependabot (weekly), npm audit, GitHub Security Advisories
GitHub Actions workflow dependencies Runtime: Node.js	Pinned action versions with known issues LTS version still in support; CVEs in the runtime itself	Dependabot Node.js Release Schedule, Vercel runtime announcements
Hosting platform: Vercel	Vendor-disclosed issues	Vercel status / changelog / advisories
Datastore: Firebase / Firestore / Google Cloud	Vendor-disclosed issues	GCP Security Bulletins
Third-party APIs (Plaid, financial data, etc.) Developer machine (macOS)	Plaid security advisories, breach disclosures OS security updates, browser updates	Vendor security pages, Plaid changelog macOS Software Update (auto-install enabled)

We do **not** run a separate web vulnerability scanner (e.g., Burp, ZAP) on the live app. The threat model — single-tenant, single-user, behind passkey-gated auth — does not warrant the noise. If scope expands beyond personal use, that decision must be revisited in this section.

2. Scanning Practices

2.1 Application dependencies

- **Dependabot** is enabled at .github/dependabot.yml for both / (Vercel root) and /backend package files. It runs **weekly** on Mondays at 09:00 America/Vancouver and opens pull requests for any flagged update with a Security Advisory or version drift.
- **GitHub Security Advisories** for the repository are subscribed to via the GitHub UI (“Security” tab). The Owner receives email notifications.
- **Manual npm audit** is run any time a dependency is added or upgraded in a non-Dependabot PR.

2.2 Runtime & platform

- The Vercel-deployed Node.js runtime is **whatever Vercel offers as default Node LTS**. The Owner reviews Vercel’s runtime announcements and advisories at least once per quarter.
- macOS auto-updates are **enabled** on the developer machine for both system and Safari/browser updates.

2.3 Production endpoint health

- The single ad-hoc test against the live deployment is the TLS posture check documented in EN-CRYPTION.md §2. This is done at least annually and after any infrastructure change.

3. Patch SLA

The Owner commits to remediating identified vulnerabilities within the following windows, measured from the time the Owner is **notified or becomes aware** (whichever comes first):

Severity (CVSS v3)	SLA	Definition
Critical (≥ 9.0)	7 days	Active exploitation likely or confirmed; remote code execution, authentication bypass, secrets disclosure.
High (7.0 – 8.9)	30 days	Significant impact; e.g., privilege escalation, denial of service against the production endpoint.
Medium (4.0 – 6.9)	90 days	Moderate impact; typically requires unusual conditions to exploit.
Low (< 4.0)	180 days	Minor or theoretical; included for completeness.

If a published patch is not yet available for a flagged vulnerability, the SLA clock pauses until the upstream vendor publishes a fix; mitigations (configuration changes, dependency removal, feature disable) are evaluated in the meantime.

If the SLA is missed, the reason is documented in `INCIDENTS.md` (created on first incident) along with the revised plan.

4. End-of-Life (EOL) Software Tracking

We treat EOL'd software as a vulnerability with a known fix (the upgrade) and apply the High SLA (30 days) to remediation.

Component	EOL source	Action on EOL
Node.js (runtime)	Node.js release schedule	Migrate to next LTS within 30 days of "End-of-Life" date for the current LTS.
npm packages	Package's own README, GitHub repo activity, or Snyk/GitHub flags	If a direct dependency is unmaintained for >12 months and has open security issues, replace or fork within 90 days.
macOS major version	Apple's deprecated-version-list	Stay on a still-supported major release (current N or N-1).
Browser (Chrome / Safari / Firefox)	Vendor release notes	Auto-update enabled; deprecated versions are not used to access the production app.

5. Process: When a Vulnerability Is Flagged

1. **Receive alert:** Dependabot PR, npm audit output, GitHub Security Advisory email, vendor disclosure.
2. **Assess severity:** confirm the CVSS score; verify the vulnerable package or version is actually used in the production code path (not just a dev dependency or unreachable transitive).

3. **Decide action:** merge the Dependabot PR, run `npm update`, replace the dependency, apply a configuration mitigation, or accept the residual risk with a note in `ACCESS_CONTROL.md` §7.
4. **Verify:** after merge/deploy, confirm the new version is in `package-lock.json` and the original alert is cleared.
5. **Log:** append a one-line note to this file's `## 7. Remediation Log` (created on first remediation) with: date, CVE / advisory ID, severity, action taken.

6. Access to Vulnerability Information

Source	Owner subscribed?
GitHub Dependabot PRs (mmfellows/wealth-navigator)	Yes — opens PRs to default branch.
GitHub Security Advisories email	Yes — sent to <code>matt@tangiblevalue.com</code> .
Plaid Security Advisories	Yes — Plaid Dashboard contact preferences.
GCP / Firebase Security Bulletins	Yes — via Google Cloud notifications for the project.
Vercel changelog / status	Reviewed manually at quarterly self-review (see <code>ACCESS_CONTROL.md</code> §6).

7. References

- `INFOSEC_POLICY.md` — parent policy
- `.github/dependabot.yml` — Dependabot configuration
- [GitHub Dependabot docs](#)
- [Node.js release schedule](#)
- [GCP Security Bulletins](#)